UNIVERSIDAD DE CONCEPCIÓN
Facultad de Ingeniería
Departamento de Ingeniería Eléctrica
Concepción, Chile

TIC3 — Taller de Integración TIC
Segundo Semestre 2026 | Prof. Radrigan
Integrantes:
• Oscar Ortiz Molina
• Joaquín Mardones Gallegos
• Gabriel Figueroa García

Sistema de Adquisición de Datos ESP32 + BMI270 Gestionado desde Raspberry Pi

Informe Técnico: Arquitectura, Decisiones de Diseño, Problemas Resueltos y Resultados

Resumen — En este proyecto desarrollamos un sistema de adquisición y telemetría de datos en tiempo real para medir aceleración y giroscopio utilizando un sensor Bosch BMI270 conectado a un microcontrolador ESP32-S3, controlado desde una Raspberry Pi. El firmware implementa un esquema en paralelo con FreeRTOS aprovechando los dos núcleos del ESP32: el Core 0 adquiere las muestras por I2C a tasa periódica estricta mientras que el Core 1 se encarga de transmitir los bloques de datos por Wi-Fi mediante sockets TCP o UDP. En la Raspberry Pi configuramos un punto de acceso propio (TIC3_RED) y desarrollamos un servidor con interfaz gráfica en PyQt5/PyQtGraph que permite cambiar parámetros (frecuencia, rango, protocolo y modo) y almacenar automáticamente las sesiones en CSV con microsegundos reales de hardware. Se detallan los problemas que enfrentamos y solucionamos (memoria de pila para el sensor, descarte de hotspot móvil por aislamiento AP y conflicto de desautenticación Wi-Fi con hostapd), comprobando que el sistema cumple el 100% de los requerimientos pedidos.

1. Introducción y Contexto

El objetivo de esta tarea fue construir una solución de adquisición remota de señales físicas conectando el mundo del hardware embebido (sensores I2C y microcontroladores) con el software de monitoreo en computadores de placa reducida (Raspberry Pi). El sistema debía ser completamente configurable desde la interfaz gráfica de la Raspberry Pi, sin tener valores fijos en el código del ESP32, y tolerando reconexiones y cambios de protocolo (TCP/UDP) sin caídas.

2. Tabla de Cumplimiento de Requisitos (RF1 a RF9)

Verificamos el cumplimiento de todos los requisitos exigidos en la pauta de evaluación:

Req.	Descripción del Requisito	Cómo se implementó	Estado
RF1	Lectura sensor 3 ejes (X, Y, Z)	Lectura de aceleración y giroscopio por I2C (SDA 42, SCL 2)	**Cumplido**
RF2	Modo de lectura secuencial	Lectura y transmisión bloqueante en una sola tarea	**Cumplido**
RF3	Modo paralelo concurrente (FreeRTOS)	Core 0 adquiere I2C a cola; Core 1 transmite por socket	**Cumplido**
RF4	Raspberry Pi emite red Wi-Fi	Punto de acceso propio 'TIC3_RED' (Canal 6, WPA2, 10.42.0.1)	**Cumplido**
RF5	Servidor en RPi, ESP32 como cliente	Servidor Python en 0.0.0.0:8080; ESP32 conecta como cliente	**Cumplido**
RF6	Protocolo TCP / UDP conmutable	TCP por defecto; cambio a UDP desde la GUI sin reiniciar placa	**Cumplido**
RF7	Parámetros configurables desde GUI	Trama JSON enviada al iniciar (ODR, rango, muestras, modo)	**Cumplido**
RF8	GUI tiempo real y guardado en sensores/	Dashboard PyQt5/PyQtGraph y guardado automático en CSV	**Cumplido**
RF9	Comparación Wireshark TCP vs UDP	Capturas pcapng analizadas: Handshake y ACKs vs Datagramas	**Cumplido**

3. Arquitectura del Sistema y Topología de Red

El sistema funciona mediante una arquitectura cliente-servidor distribuida en dos equipos principales:

Esquema General de Conexión:
• **Sensor Inercial:** Bosch BMI270 (acelerómetro y giroscopio de 6 ejes) conectado por bus I2C (SDA=GPIO 42, SCL=GPIO 2, 400 kHz).
• **Microcontrolador Embebido:** ESP32-S3 programado en C nativo sobre ESP-IDF v6.1 con FreeRTOS.
• **Red Wi-Fi:** Red propia emitida por la Raspberry Pi con SSID **TIC3_RED**, seguridad WPA2-PSK (clave: *raspi123*) en Canal 6. La Raspberry Pi toma la IP fija **10.42.0.1** y el ESP32 recibe automáticamente la IP **10.42.0.30** por DHCP.
• **Servidor y Visualización:** Raspberry Pi 5 corriendo el servidor multi-socket en Python y la interfaz gráfica.

4. Decisiones de Diseño en el Firmware del ESP32

Para lograr que el muestreo fuera estable y que la red no perjudicara la adquisición física del sensor, implementamos las siguientes decisiones:

4.1. Modo Paralelo con FreeRTOS en Doble Núcleo (RF3)

A frecuencias medias y altas (de 100 Hz a 1600 Hz), el envío por red inalámbrica puede tener pequeños retrasos debido a retransmisiones o ruido en el canal. Si la lectura del sensor y el envío por Wi-Fi se hacen juntos en el mismo hilo, la lectura se frena y se pierde la periodicidad. Para evitar esto, separamos las tareas en los dos núcleos del ESP32-S3:
• **Núcleo 0 (Productor):** Ejecuta la tarea task_parallel_producer. Su único trabajo es leer los 12 bytes del sensor BMI270 en el microsegundo exacto correspondiente al periodo configurado (usando esp_timer_get_time() y vTaskDelayUntil()) y guardar la lectura en una cola de FreeRTOS (xQueue).
• **Núcleo 1 (Consumidor):** Ejecuta la tarea task_parallel_consumer. Extrae los datos de la cola, los agrupa en el búfer y los transmite por el socket hacia la Raspberry Pi. Así, si el Wi-Fi tarda un poco más en enviar, el Core 0 sigue adquiriendo a tiempo sin perder muestras.

4.2. Modo Secuencial (RF2)

En este modo se ejecuta una sola tarea lineal que lee el sensor y manda los datos directamente por el socket de forma bloqueante. Esto nos sirvió para comparar ambos modos y demostrar en las pruebas cómo la latencia del socket afecta la tasa efectiva de muestreo.

4.3. Empaquetamiento en Bloques de 1 Segundo

Si el ESP32 enviara cada muestra individualmente a 400 Hz o 1600 Hz, la red se saturaría de inmediato porque cada paquete pequeño llevaría cabeceras de red que pesan más que los datos útiles. Para solucionarlo, acumulamos todas las lecturas tomadas durante 1 segundo exacto y las enviamos juntas en un solo paquete JSON. Esto hace que la transmisión sea mucho más liviana y ordenada.

4.4. Configuración Dinámica por JSON (RF7)

Cuando el ESP32 se conecta al Wi-Fi, envía un saludo inicial avisando que está listo y se queda esperando la orden de la Raspberry Pi. La interfaz le envía una trama JSON como esta:
{"cmd":"start","odr":400,"range":8,"samples":2000,"protocol":"UDP","mode":"parallel"}
El microcontrolador lee estos valores, reconfigura al vuelo los registros del sensor (frecuencia ODR y rango de aceleración) y arranca la adquisición sin tener que reiniciar ni reprogramar la placa.

5. Servidor y Persistencia en la Raspberry Pi

El servidor corre directamente en la Raspberry Pi 5 y fue diseñado para ser robusto y fácil de usar:

5.1. Manejo de Red Multi-Socket y Tolerancia a Fallos

El servidor escucha en el puerto 8080 en todas las interfaces de red (0.0.0.0) para funcionar tanto con el Hotspot como en la red del laboratorio. Utiliza la opción SO_REUSEADDR a nivel de socket, lo que permite liberar el puerto inmediatamente cuando se detiene una sesión o se cambia de puerto, evitando el error de que el puerto quede ocupado. Además, la recepción de datos corre en un hilo secundario con select() no bloqueante para que la ventana de la aplicación nunca se pegue.

5.2. Guardado en sensores/ con Timestamps Reales

El enunciado pide que cada archivo en sensores/ incluya timestamp, modo de lectura, frecuencia ODR, rango y los valores de los ejes. Para que los datos no tuvieran marcas de tiempo repetidas por bloque, calculamos el timestamp de cada fila usando la hora de inicio en el reloj del sistema más los microsegundos exactos medidos por el temporizador de hardware del ESP32:
timestamp_i = t_inicio + (t_hardware_i - t_hardware_0)
De esta manera, el archivo CSV resultante tiene incrementos de tiempo reales y parejos (10.0 ms para 100 Hz, 2.5 ms para 400 Hz y 0.625 ms para 1600 Hz).

6. Interfaz Gráfica de Telemetría (PyQt5 + PyQtGraph)

Al principio desarrollamos una interfaz básica con Tkinter y Matplotlib. Sin embargo, al probar frecuencias más altas (400 Hz y más), Matplotlib consumía casi todo el procesador de la Raspberry Pi (>85% CPU) y las gráficas se veían a tirones. Por esto migramos a **PyQt5 con PyQtGraph**, una librería especializada en graficación científica en tiempo real que maneja los datos con matrices de NumPy y aceleración de Qt. Con esto logramos refrescar las curvas a 25 cuadros por segundo consumiendo menos de un 8% de CPU.



Figura 1: *Interfaz gráfica funcionando en tiempo real en la Raspberry Pi, adquiriendo telemetría a 400 Hz vía TCP.*

Componentes de la Interfaz:
• **Píldoras de Estado (Header):** Muestran si el servidor está escuchando o adquiriendo, la IP del ESP32 conectado (10.42.0.30) y la IP de la Raspberry Pi.
• **Tarjetas KPI:** Tres recuadros con números grandes para leer desde el proyector: total de muestras, tasa real en Hz calculada por hardware y bloques de 1 s recibidos.
• **Gráficas de Tiempo:** Dos gráficos independientes con fondo oscuro: Aceleración Lineal en m/s² (arriba) y Velocidad Angular en rad/s (abajo), con curvas en Cyan (X), Coral (Y) y Ámbar (Z).

7. Problemas Encontrados y Soluciones Aplicadas

Durante el desarrollo del trabajo nos encontramos con tres problemas técnicos importantes que tuvimos que diagnosticar y resolver para que el sistema funcionara de forma estable:

7.1. Desbordamiento de Memoria (Stack Overflow) al Iniciar el Sensor BMI270

• **Problema:** Al compilar y probar la lectura I2C, el ESP32 se caía inmediatamente y se reiniciaba arrojando un error de pánico de kernel (*Guru Meditation Error*) justo cuando ejecutaba la función de inicialización del sensor.

• **Causa:** Para funcionar, el sensor Bosch BMI270 requiere cargar un archivo de microcódigo de inicialización que pesa **8 KB**. La tarea principal app_main en la configuración por defecto de ESP-IDF viene con solo **3.5 KB** de pila (stack). Al reservar el búfer en memoria para escribir por I2C, la pila se desbordaba.

• **Solución:** Modificamos la configuración del proyecto en el archivo sdkconfig.defaults agregando la línea CONFIG_ESP_MAIN_TASK_STACK_SIZE=16384 para asignarle 16 KB de pila a la tarea principal. Con ese espacio extra, el búfer cargó correctamente y el sensor respondió con su identificador de chip (CHIPID: 0x24).

7.2. Intento con Hotspot de Celular Bloqueado por Aislamiento de Clientes

• **Problema:** Mientras configurábamos la red de la Raspberry Pi, probamos usar el punto de acceso (hotspot) de un celular para conectar ambos equipos a una misma red. Aunque tanto el ESP32 como la Raspberry Pi se conectaron y recibieron IP, el ESP32 no lograba conectarse al socket del servidor y los paquetes de ping se perdían al 100%.

• **Causa:** La mayoría de los celulares con Android o iOS tienen activada por seguridad una función llamada *Aislamiento de Clientes (AP Isolation)*. Esta función permite que cada dispositivo conectado al teléfono tenga internet, pero bloquea todo el tráfico directo entre los dispositivos conectados entre sí.

• **Solución:** Descartamos completamente el teléfono y nos concentramos en configurar el punto de acceso propio en la Raspberry Pi, lo que además cumple de forma estricta con el requerimiento RF4 de la pauta.

7.3. Desconexión Wi-Fi a los 20 ms en la Red de la Raspberry Pi (Razones 20 y 13)

• **Problema:** Al activar la red *TIC3_RED* en la Raspberry Pi, el ESP32 alcanzaba a asociarse y pedía IP por DHCP, pero a los 20 milisegundos se desconectaba bruscamente recibiendo una orden de expulsión (deauth) desde la Raspberry Pi.

• **Causa:** Revisando los registros del sistema con journalctl descubrimos dos causas: 1) Había un servicio antiguo de hostapd.service corriendo como demonio en segundo plano que interfería con la tarjeta inalámbrica (Razón 20); 2) La radio de la Raspberry Pi no tenía definido el código de país de Chile (CL), lo que provocaba discrepancias en los canales permitidos (Razón 13).

• **Solución:** Enmascaramos el servicio en conflicto (sudo systemctl mask hostapd), configuramos el país en Chile mediante raspi-config y configuramos la red con NetworkManager fijando el Canal 6 con WPA2-PSK y clave *raspi123*. Tras aplicar esto, la conexión del ESP32 fue instantánea y permanente.

8. Resultados Experimentales y Análisis de Parámetros

En el laboratorio realizamos pruebas con distintas combinaciones de frecuencia de muestreo, sensibilidad del acelerómetro, modo de trabajo y protocolo. A continuación se presentan los resultados de las tres sesiones guardadas en el repositorio:

Archivo de Sesión	ODR	Rango	Modo	Protocolo	Muestras	Tasa Medida	Resultado
sesion_..._152423	100 Hz	±8g	Paralelo (FreeRTOS)	TCP	1.000	100.0 Hz	100% recibidas (0% pérdidas)
sesion_..._152520	400 Hz	±4g	Paralelo (FreeRTOS)	UDP	2.000	400.1 Hz	100% recibidas (0% pérdidas)
sesion_..._152558	1600 Hz	±16g	Secuencial	TCP	5.000	987.2 Hz	1600 Hz inicial; frena por socket bloqueante

8.1. Efecto del Rango y Sensibilidad del BMI270 (±2g a ±16g)

El sensor convierte la aceleración con un conversor analógico-digital (ADC) de 16 bits. El rango seleccionado cambia la resolución por cada bit:

• **En rango ±2g (16384 cuentas por g):** La sensibilidad es máxima y el nivel de ruido es muy bajo. Sirve para captar inclinaciones y movimientos lentos con gran detalle. Sin embargo, como 1g es la gravedad normal en reposo (que ocupa la mitad de la escala), basta mover la placa con algo de fuerza para que el sensor se sature y corte la señal.

• **En rango ±16g (2048 cuentas por g):** El rango medible es 8 veces más amplio. Es ideal para sacudidas fuertes, caídas o impactos, ya que no se satura fácilmente. A cambio, el paso de medición es un poco más grueso, por lo que para movimientos muy sutiles se nota menos resolución.

8.2. Efecto de la Frecuencia de Muestreo (ODR)

Al cambiar la frecuencia en la interfaz, el filtro digital interno del sensor adapta su ancho de banda:

• A **100 Hz** las curvas son limpias y estables, adecuadas para verificar la gravedad en reposo y rotaciones básicas.

• A **400 Hz** se capturan vibraciones rápidas y transitorios con excelente nitidez sin sobrecargar la red.

• A **1600 Hz (Modo Secuencial):** En el primer bloque de 1 segundo el sensor muestrea a ~1600 Hz, pero al operar en modo secuencial bloqueante, la pausa de envío por el socket TCP (~0.65 s entre cada bloque de 1600 muestras) hace que la tasa efectiva acumulada disminuya hasta ~988 Hz (987.2 Hz en el CSV), comprobando en la práctica por qué el modo paralelo con FreeRTOS es necesario a alta frecuencia.

9. Comparación de Tráfico con Wireshark: TCP vs UDP (RF9)

Grabamos el tráfico con tcpdump en la Raspberry Pi y abrimos las capturas en Wireshark (wireshark_tcp.pcapng y wireshark_udp.pcapng), notando diferencias claras entre ambos protocolos:

Aspecto	TCP (Confiable por defecto)	UDP (Rápido por datagramas)
Inicio de sesión	Requiere Handshake de 3 vías (SYN, SYN-ACK, ACK) antes de mandar datos.	Sin conexión previa. El primer paquete ya contiene datos del sensor.
Confirmaciones	La Raspberry Pi responde con un paquete ACK por cada bloque recibido.	El ESP32 transmite sin esperar ninguna confirmación de la Raspberry Pi.
Cabecera	Cabecera de 20 a 32 bytes con números de secuencia y control de flujo.	Cabecera fija y muy liviana de exactamente 8 bytes.
Comportamiento	Garantiza que no se pierdan datos; retransmite si hay interferencia.	Menor latencia de llegada; si un paquete se pierde, no se recupera.

10. Conclusiones

A partir de las pruebas y el desarrollo del sistema, llegamos a las siguientes conclusiones:

1. **El modo paralelo con FreeRTOS fue fundamental:** Separar la lectura I2C en el Core 0 y el socket en el Core 1 permitió que las muestras se tomaran en el instante exacto, evitando que la lentitud o retransmisiones del Wi-Fi frenaran al sensor.
2. **El envío por bloques de 1 segundo salvó el ancho de banda:** Acumular las muestras y mandarlas en un paquete JSON en vez de muestra a muestra redujo drásticamente las cabeceras de red y permitió alcanzar frecuencias altas sin colapsar el punto de acceso.
3. **La migración a PyQtGraph solucionó el rendimiento visual:** El cambio de Matplotlib a PyQtGraph demostró la importancia de usar librerías con aceleración gráfica en computadores embebidos como la Raspberry Pi, logrando graficar a 25 cuadros por segundo sin sobrecargar la CPU.
4. **Consistencia en los archivos CSV:** Calcular los timestamps sumando los microsegundos del hardware del ESP32 aseguró que cada registro guardado en sensores/ tenga incrementos de tiempo reales y ordenados.

11. Lecciones Aprendidas

• **Memoria en sistemas embebidos:** Aprendimos que los sensores modernos que cargan archivos de firmware o microcódigo (como los 8 KB del BMI270) requieren revisar con cuidado el tamaño de pila en ESP-IDF y no confiarse de los valores por defecto.
• **Diagnóstico de redes con herramientas reales:** El uso de Wireshark y la revisión de los logs del sistema en Linux nos permitieron entender por qué fallaba el hotspot del celular (aislamiento AP) y cómo resolver los conflictos de red en la Raspberry Pi mirando los códigos de razón de desconexión.
• **Organización y control de versiones:** Llevar el repositorio con Git y documentar los prompts en prompts.md nos ayudó a mantener un registro claro de todo el proceso de trabajo en equipo.